

编译原理

第六讲：语法分析之预测分析法

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 4 月 7 日



福州大学
FUZHOU UNIVERSITY 1958



① 预备知识

② LL(1) 分析法

③ 小结

目录

- ① 预备知识
- ② LL(1) 分析法
- ③ 小结



下推自动机

下推自动机（Pushdown Automaton, PDA）是一个抽象的机器用于识别和接受字符输入。它在非确定型有限自动机的基础上添加了一个无限规模的栈。它的数学定义为一个六元组：

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

下推自动机

下推自动机（Pushdown Automaton, PDA）是一个抽象的机器用于识别和接受字符输入。它在非确定型有限自动机的基础上添加了一个无限规模的栈。它的数学定义为一个六元组：

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q 是所有状态的集合，集合的大小有限。

下推自动机

下推自动机（Pushdown Automaton, PDA）是一个抽象的机器用于识别和接受字符输入。它在非确定型有限自动机的基础上添加了一个无限规模的栈。它的数学定义为一个六元组：

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q 是所有状态的集合，集合的大小有限。
- Σ 是输入字符的集合，集合的大小有限。特别地， $\Sigma_\epsilon = \{\epsilon\} \cup \Sigma$

下推自动机

下推自动机（Pushdown Automaton, PDA）是一个抽象的机器用于识别和接受字符输入。它在非确定型有限自动机的基础上添加了一个无限规模的栈。它的数学定义为一个六元组：

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q 是所有状态的集合，集合的大小有限。
- Σ 是输入字符的集合，集合的大小有限。特别地， $\Sigma_\epsilon = \{\epsilon\} \cup \Sigma$
- Γ 是栈字符的集合，大小有限。特别地， $\Gamma_\epsilon = \{\epsilon\} \cup \Gamma$

下推自动机

下推自动机 (Pushdown Automaton, PDA) 是一个抽象的机器用于识别和接受字符输入。它在非确定型有限自动机的基础上添加了一个无限规模的栈。它的数学定义为一个六元组:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q 是所有状态的集合, 集合的大小有限。
- Σ 是输入字符的集合, 集合的大小有限。特别地, $\Sigma_\epsilon = \{\epsilon\} \cup \Sigma$
- Γ 是栈字符的集合, 大小有限。特别地, $\Gamma_\epsilon = \{\epsilon\} \cup \Gamma$
- δ 是转移函数

$$\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow Q \times \Gamma_\epsilon$$

即同时根据输入和当前栈顶元素进行转移, 在改变状态的同时替换栈顶元素。

下推自动机

下推自动机 (Pushdown Automaton, PDA) 是一个抽象的机器用于识别和接受字符输入。它在非确定型有限自动机的基础上添加了一个无限规模的栈。它的数学定义为一个六元组:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q 是所有状态的集合, 集合的大小有限。
- Σ 是输入字符的集合, 集合的大小有限。特别地, $\Sigma_\epsilon = \{\epsilon\} \cup \Sigma$
- Γ 是栈字符的集合, 大小有限。特别地, $\Gamma_\epsilon = \{\epsilon\} \cup \Gamma$
- δ 是转移函数

$$\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow Q \times \Gamma_\epsilon$$

即同时根据输入和当前栈顶元素进行转移, 在改变状态的同时替换栈顶元素。

- q_0, F 的意思和 NFA 中的一致。

定理

一个语言是上下文无关的，当且仅当存在一个下推自动机识别该语言。

引理

若一个语言是上下文无关的，则存在一个下推自动机识别能够识别该语言。

引理

若一个语言可以被 PDA 接受，它便是一个上下文无关语言。

下推自动机与上下文无关文法

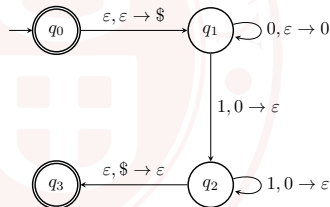
上下文无关文法

$$S \rightarrow 0S1 | \varepsilon$$

上下文无关语言

$$\{0^n 1^n | n \geq 0\}$$

下推自动机



下推自动机与上下文无关文法

上下文无关文法

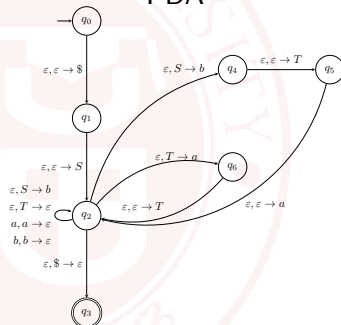
$$S \rightarrow aTb|b$$

$$T \rightarrow Ta|\varepsilon$$

上下文无关语言

$$\{a^n b | n \geq 0\}$$

PDA



扩展知识之 PEG

解析表达式文法 (Parsing Expression Grammar) 是一种描述语法规则的形式化方法，专为解析而设计。相比 CFG，PEG 更关注如何通过规则匹配输入文本，而非生成语言结构。

两种方法并非完全对立，现代工具（如 ANTLR 4）允许混合使用 CFG 与 PEG 特性（如优先规则）。

例

```
Expr ← AddExpr
AddExpr ← MulExpr (('+' / '-') MulExpr)*
MulExpr ← Primary (('*' / '/') Primary)*
Primary ← Number / '(' Expr ') '
Number ← [0-9]+
```

PEG 与 CFG 的核心差异对比

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

扩展知识：语法分析的最坏情况

语法分析过程的最坏时间复杂度是多少？



扩展知识：语法分析的最坏情况

语法分析过程的最坏时间复杂度是多少？

- $O(n^3)$



扩展知识：语法分析的最坏情况

语法分析过程的最坏时间复杂度是多少？

- $O(n^3)$
- 如何证明？
 - ① 改写文法为 $A \rightarrow BC$;



扩展知识：语法分析的最坏情况

语法分析过程的最坏时间复杂度是多少？

- $O(n^3)$
- 如何证明？
 - ① 改写文法为 $A \rightarrow BC$;
 - ② 动态规划 `bool parsing(int l, int r);`

扩展知识之泵引理

正则语言的泵引理

$\{0^n 1^n | n \geq 1\}$ 不是正则语言.

假设 $\{0^n 1^n | n \geq 1\}$ 是正则, 则存在 p (泵引理常数) 使得泵引理成立。

取字符串 $w = 0^p 1^p$, 尝试将其划分为三个不重叠的子串 x, y, z , 使其满足泵引理的条件: $|xy| \leq p \wedge |y| \geq 1 \wedge \forall k \geq 0 : xy^k z \in L$ 。

会发现无论如何划分, $xy^0 z = xz$ 都会应为 0 的数量比 1 少而不属于 L 。

这与泵引理冲突, 因此 $\{0^n 1^n | n \geq 1\}$ 不是正则语言。 □

扩展知识之泵引理

引理 (上下文无关语言的泵引理)

L 是下文无关语言, 则存在整数 $n \geq 1$ 使得任意长度超过 n 的字符串 w 都可以被改写成 $uvxyz$ 的形式并满足:

- ① $|vxy| \leq n$;
- ② $|vy| \geq 1$;
- ③ $\forall k \geq 0 : uv^kxy^kz \in L$ 。

语法分析器的类型

自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从根开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从输入串开始，逐步进行归约，直到文法的开始符号。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从叶节点开始，构造语法树。
- 具体实现：算符优先分析法、LR 分析法。

目录

① 预备知识

② LL(1) 分析法

LL(1) 文法概述

表驱动的预测分析法

构造 LL(1) 分析表

③ 小结



目录

① 预备知识

② LL(1) 分析法

LL(1) 文法概述

表驱动的预测分析法

构造 LL(1) 分析表

③ 小结



LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；

LL(1) 文法

定义

- LL(**1**) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - **1** 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。

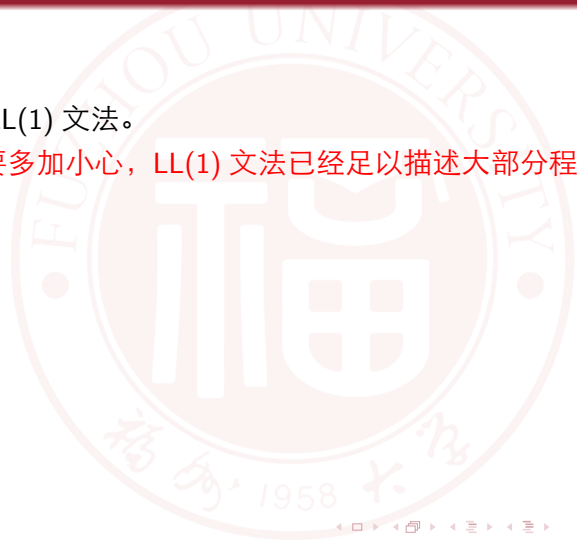
关于 LL(1) 文法的说明

- 左递归文法和二义文法都不是 LL(1) 文法。



关于 LL(1) 文法的说明

- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。



关于 LL(1) 文法的说明

- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ε 边作为最后的默认选择以解决 ε 的二义性（例如，if 语句的 else 匹配问题）。

关于 LL(1) 文法的说明

- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ε 边作为最后的默认选择以解决 ε 的二义性（例如，if 语句的 else 匹配问题）。
- LL(1) 文法具有唯一性，对于每个输入串，LL(1) 分析器只能有一个有效的解析树。

关于 LL(1) 文法的说明

- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ε 边作为最后的默认选择以解决 ε 的二义性（例如，if 语句的 else 匹配问题）。
- LL(1) 文法具有唯一性，对于每个输入串，LL(1) 分析器只能有一个有效的解析树。
- LL(1) 文法适合于自顶向下的语法分析方法，如递归下降分析和预测分析。

目录

① 预备知识

② LL(1) 分析法

LL(1) 文法概述

表驱动的预测分析法

构造 LL(1) 分析表

③ 小结



表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 小提问：对于每一个 NFA 都可以构造一个 DFA 与其等价，那所有的 PDA 都可以构造一个确定性的下推自动机与其等价吗？

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。
- 要求：文法是 LL(1) 的。只有对于 LL(1) 文法，可以对输入串进行有效的无回溯的自上而下分析。



预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号\$）；

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号\$）；
 - 每个单元格里填写对应的产生式，或者错误标记；

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号\$）；
 - 每个单元格里填写对应的产生式，或者错误标记；
 - 单元格 (i, j) 的内容表示若在匹配第 i 行的非终结符时遇到第 j 列的终结符时，该确定性地选用的生成式。

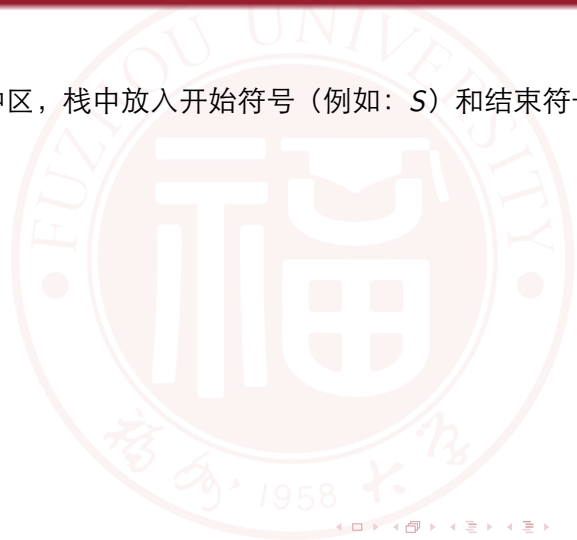
预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。



预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如：S）和结束符号\$。
- ② 重复执行此步：



预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。

预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。
 - 如果 X 是一个终结符或 $\$$ ：

预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。
 - 如果 X 是一个终结符或 $\$$ ：
 - 如果 $X = a$ ，则弹出栈顶符号，并将输入指针向右移动一个位置。

预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。
 - 如果 X 是一个终结符或 $\$$ ：
 - 如果 $X = a$ ，则弹出栈顶符号，并将输入指针向右移动一个位置。
 - 否则，报告错误并停止分析。

预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。
 - 如果 X 是一个终结符或 $\$$ ：
 - 如果 $X = a$ ，则弹出栈顶符号，并将输入指针向右移动一个位置。
 - 否则，报告错误并停止分析。
 - 如果 X 是一个非终结符：

预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。
 - 如果 X 是一个终结符或 $\$$ ：
 - 如果 $X = a$ ，则弹出栈顶符号，并将输入指针向右移动一个位置。
 - 否则，报告错误并停止分析。
 - 如果 X 是一个非终结符：
 - 查预测分析表 $M[X, a]$ 。

预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。
 - 如果 X 是一个终结符或 $\$$ ：
 - 如果 $X = a$ ，则弹出栈顶符号，并将输入指针向右移动一个位置。
 - 否则，报告错误并停止分析。
 - 如果 X 是一个非终结符：
 - 查预测分析表 $M[X, a]$ 。
 - 如果表项为空，报告错误并停止分析。

预测分析器的控制算法

- ① 初始化：将输入串放在输入缓冲区，栈中放入开始符号（例如： S ）和结束符号 $\$$ 。
- ② 重复执行此步：
 - 设栈顶符号为 X ，当前输入符号为 a 。
 - 如果 X 是一个终结符或 $\$$ ：
 - 如果 $X = a$ ，则弹出栈顶符号，并将输入指针向右移动一个位置。
 - 否则，报告错误并停止分析。
 - 如果 X 是一个非终结符：
 - 查预测分析表 $M[X, a]$ 。
 - 如果表项为空，报告错误并停止分析。
 - 否则，弹出栈顶符号，并将产生式右部的符号（如果有）按逆序压入栈中。

目录

① 预备知识

② LL(1) 分析法

LL(1) 文法概述

表驱动的预测分析法

构造 LL(1) 分析表

③ 小结



构造 LL(1) 分析表的步骤

- ① 消除左递归：将文法中的左递归消除掉，以满足 LL(1) 文法的要求。



构造 LL(1) 分析表的步骤

- ① 消除左递归：将文法中的左递归消除掉，以满足 LL(1) 文法的要求。
- ② 提取左公共因子：如果一个非终结符的多个产生式有相同的前缀，需要提取出公共部分，以避免预测分析表中的冲突。

构造 LL(1) 分析表的步骤

- ❶ 消除左递归：将文法中的左递归消除掉，以满足 LL(1) 文法的要求。
- ❷ 提取左公共因子：如果一个非终结符的多个产生式有相同的前缀，需要提取出公共部分，以避免预测分析表中的冲突。
- ❸ 计算 FIRST 集：对于每个非终结符，计算其 FIRST 集，即可以从该非终结符推导出的字符串的第一个符号集合。

构造 LL(1) 分析表的步骤

- ❶ 消除左递归：将文法中的左递归消除掉，以满足 LL(1) 文法的要求。
- ❷ 提取左公共因子：如果一个非终结符的多个产生式有相同的前缀，需要提取出公共部分，以避免预测分析表中的冲突。
- ❸ 计算 FIRST 集：对于每个非终结符，计算其 FIRST 集，即可以从该非终结符推导出的字符串的第一个符号集合。
- ❹ 计算 FOLLOW 集：对于每个非终结符，计算其 FOLLOW 集，即在某个产生式中，该非终结符后面可能出现的符号集合。

构造 LL(1) 分析表的步骤

- ① 消除左递归：将文法中的左递归消除掉，以满足 LL(1) 文法的要求。
- ② 提取左公共因子：如果一个非终结符的多个产生式有相同的前缀，需要提取出公共部分，以避免预测分析表中的冲突。
- ③ 计算 FIRST 集：对于每个非终结符，计算其 FIRST 集，即可以从该非终结符推导出的字符串的第一个符号集合。
- ④ 计算 FOLLOW 集：对于每个非终结符，计算其 FOLLOW 集，即在某个产生式中，该非终结符后面可能出现的符号集合。
- ⑤ 填充预测分析表：根据文法规则、FIRST 集和 FOLLOW 集，填充预测分析表中的相应单元格。

FIRST

定义

$\text{FIRST}(\alpha)$ 是可以由 α 推导得到的串首符号的集合, 其中 α 是任意的文法符号串, 具体地

- $\forall \alpha \in (\mathbb{V}_T \cup \mathbb{V}_N)^+, \text{FIRST}(\alpha) = \{a | \alpha \Rightarrow^* a\beta, a \in \mathbb{V}_T, \beta \in (\mathbb{V}_T \cup \mathbb{V}_N)^*\}$
- 若 $\alpha \Rightarrow^* \varepsilon$ 合法, 则 $\varepsilon \in \text{FIRST}(\alpha)$

推论

如果非终结符 A 的所有候选式的首终结符集两两不相交, 当要求 A 匹配输入串时, A 就能根据当前输入符号 c , 选择唯一的正确的候选式, 即串首终结符集中包含 c 的那个候选式。

FIRST 集练习 1: 基础分支与终结符

例

计算文法 G_1 的所有非终结符的 FIRST 集:

文法 G_1

$$S \rightarrow aAS \mid b$$

$$A \rightarrow bA \mid \varepsilon$$

$$B \rightarrow cS \mid d$$

FIRST 集练习 1: 基础分支与终结符

例

计算文法 G_1 的所有非终结符的 FIRST 集:

文法 G_1

$$S \rightarrow aAS \mid b$$

$$A \rightarrow bA \mid \varepsilon$$

$$B \rightarrow cS \mid d$$

- $\text{FIRST}(A) = \{b, \varepsilon\}$

FIRST 集练习 1: 基础分支与终结符

例

计算文法 G_1 的所有非终结符的 FIRST 集:

文法 G_1

$$S \rightarrow aAS \mid b$$

$$A \rightarrow bA \mid \varepsilon$$

$$B \rightarrow cS \mid d$$

- $\text{FIRST}(A) = \{b, \varepsilon\}$
- $\text{FIRST}(S) = \{a, b\}$
(由 aAS 贡献 a , 由 b 贡献 b)

FIRST 集练习 1: 基础分支与终结符

例

计算文法 G_1 的所有非终结符的 FIRST 集:

文法 G_1

$$S \rightarrow aAS \mid b$$

$$A \rightarrow bA \mid \varepsilon$$

$$B \rightarrow cS \mid d$$

- $\text{FIRST}(A) = \{b, \varepsilon\}$
- $\text{FIRST}(S) = \{a, b\}$
(由 aAS 贡献 a , 由 b 贡献 b)
- $\text{FIRST}(B) = \{c, d\}$

FIRST 集练习 1: 基础分支与终结符

例

计算文法 G_1 的所有非终结符的 FIRST 集:

文法 G_1

$$S \rightarrow aAS \mid b$$

$$A \rightarrow bA \mid \varepsilon$$

$$B \rightarrow cS \mid d$$

- $\text{FIRST}(A) = \{b, \varepsilon\}$
- $\text{FIRST}(S) = \{a, b\}$
(由 aAS 贡献 a , 由 b 贡献 b)
- $\text{FIRST}(B) = \{c, d\}$

解析要点

当产生式右部以**终结符**开头时, 该终结符直接加入左部的 FIRST 集。

FIRST 集练习 2: ε -穿透与链式推导

例

计算文法 G_2 的 FIRST 集 (注意 F 的 ε 产生式):

$$E \rightarrow TR$$

$$R \rightarrow +TR \mid \varepsilon$$

$$T \rightarrow FY$$

$$Y \rightarrow *FY \mid \varepsilon$$

$$F \rightarrow (E) \mid id \mid \varepsilon$$

FIRST 集练习 2: ε -穿透与链式推导

例

计算文法 G_2 的 FIRST 集 (注意 F 的 ε 产生式):

$$E \rightarrow TR$$

$$R \rightarrow +TR \mid \varepsilon$$

$$T \rightarrow FY$$

$$Y \rightarrow *FY \mid \varepsilon$$

$$F \rightarrow (E) \mid id \mid \varepsilon$$

- $\text{FIRST}(F) = \{ (, id, \varepsilon \}$

FIRST 集练习 2: ε -穿透与链式推导

例

计算文法 G_2 的 FIRST 集 (注意 F 的 ε 产生式):

$$E \rightarrow TR$$

$$R \rightarrow +TR \mid \varepsilon$$

$$T \rightarrow FY$$

$$Y \rightarrow *FY \mid \varepsilon$$

$$F \rightarrow (E) \mid id \mid \varepsilon$$

- $\text{FIRST}(F) = \{ (, id, \varepsilon \}$
- $\text{FIRST}(Y) = \{ *, \varepsilon \}$

FIRST 集练习 2: ε -穿透与链式推导

例

计算文法 G_2 的 FIRST 集 (注意 F 的 ε 产生式):

$$E \rightarrow TR$$

$$R \rightarrow +TR \mid \varepsilon$$

$$T \rightarrow FY$$

$$Y \rightarrow *FY \mid \varepsilon$$

$$F \rightarrow (E) \mid id \mid \varepsilon$$

- $\text{FIRST}(F) = \{ (, id, \varepsilon \}$
- $\text{FIRST}(Y) = \{ *, \varepsilon \}$
- $\text{FIRST}(T) = \{ (, id, *, \varepsilon \}$
逻辑: 因为 $F \Rightarrow^* \varepsilon$, 故为
 $(\text{FIRST}(F) \setminus \{ \varepsilon \}) \cup \text{FIRST}(Y)$

FIRST 集练习 2: ε -穿透与链式推导

例

计算文法 G_2 的 FIRST 集 (注意 F 的 ε 产生式):

$$E \rightarrow TR$$

$$R \rightarrow +TR \mid \varepsilon$$

$$T \rightarrow FY$$

$$Y \rightarrow *FY \mid \varepsilon$$

$$F \rightarrow (E) \mid id \mid \varepsilon$$

- $\text{FIRST}(F) = \{ (, id, \varepsilon \}$
- $\text{FIRST}(Y) = \{ *, \varepsilon \}$
- $\text{FIRST}(T) = \{ (, id, *, \varepsilon \}$
逻辑: 因为 $F \Rightarrow^* \varepsilon$, 故为
 $(\text{FIRST}(F) \setminus \{ \varepsilon \}) \cup \text{FIRST}(Y)$
- $\text{FIRST}(R) = \{ +, \varepsilon \}$

FIRST 集练习 2: ε -穿透与链式推导

例

计算文法 G_2 的 FIRST 集 (注意 F 的 ε 产生式):

$$E \rightarrow TR$$

$$R \rightarrow +TR \mid \varepsilon$$

$$T \rightarrow FY$$

$$Y \rightarrow *FY \mid \varepsilon$$

$$F \rightarrow (E) \mid id \mid \varepsilon$$

- $\text{FIRST}(F) = \{ (, id, \varepsilon \}$
- $\text{FIRST}(Y) = \{ *, \varepsilon \}$
- $\text{FIRST}(T) = \{ (, id, *, \varepsilon \}$
 逻辑: 因为 $F \Rightarrow^* \varepsilon$, 故为
 $(\text{FIRST}(F) \setminus \{ \varepsilon \}) \cup \text{FIRST}(Y)$
- $\text{FIRST}(R) = \{ +, \varepsilon \}$
- $\text{FIRST}(E) = \{ (, id, *, +, \varepsilon \}$
 逻辑: 因为 $T \Rightarrow^* \varepsilon$, 故为
 $(\text{FIRST}(T) \setminus \{ \varepsilon \}) \cup \text{FIRST}(R)$

计算 FIRST 的算法

Algorithm 1: 计算语法符号 X 的串首集 $\text{FIRST}(X)$ 的算法

Input: 语法符号 X

Output: 串首集 $\text{FIRST}(X)$

```

1  if  $X \in \mathbb{V}^T$  then
2     $\text{FIRST}(X) \leftarrow \{X\};$ 
3  else
4     $\text{FIRST}(X) \leftarrow \emptyset;$ 
5    for all  $X \rightarrow Y_1 Y_2 \dots Y_k \in \mathbb{P}$  do
6      for  $i \leftarrow 1$  to  $k$  do
7        if  $a \in \text{FIRST}(Y_i) \wedge \forall j < i, \varepsilon \in \text{FIRST}(Y_j)$  then
8           $\text{FIRST}(X) \leftarrow \text{FIRST}(X) \cup \{a\};$ 
9        end
10       end
11       if  $\forall j \leq i, \varepsilon \in \text{FIRST}(Y_j)$  then
12         end
13     end
14 end
  
```

FOLLOW

- 对于非终结符号 A , $\text{FOLLOW}(A)$ 是所有在某些句型中紧跟在 A 右边的终结符号的集合。
- A 与 a 之间可能存在其他的非终结符, 但都会随着推导变成 ε 。
- 特别地, 如果 A 是某些句型最右的符号, 则 $\$ \in \text{FOLLOW}(A)$, 其中 $\$$ 表示输入的终止 (EOF)。

例

如果存在 $S \Rightarrow^* \alpha A a \beta$ 的推导, 则 $a \in \text{FOLLOW}(A)$ 。

计算 FOLLOW 的算法

Algorithm 2: 计算所有非终结符紧随集 FOLLOW 的算法

Input: 语法符号 A

Output: 紧随集 FOLLOW

```
1 FOLLOW( $S$ )  $\rightarrow$   $\{\$ \}$ ;  
2 repeat  
3   对于所有产生式判定如下条件:  
4   if  $A \rightarrow \alpha B \beta \in \mathbb{P}$  then  
5     FOLLOW( $B$ )  $\leftarrow$  FOLLOW( $B$ )  $\cup$  (FIRST( $\beta$ )  $\setminus$   $\{\epsilon\}$ );  
6   end  
7   if  $A \rightarrow \alpha B \in \mathbb{P}$  or  $A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \epsilon \in \text{FIRST}(\beta)$  then  
8     FOLLOW( $B$ )  $\leftarrow$  FOLLOW( $B$ )  $\cup$  FOLLOW( $A$ );  
9   end  
10 until 所有非终结符的紧随集不再发生变化;
```

计算文法符号 X 的 FOLLOW(X)

例

计算下列文法的 FOLLOW 集:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

计算文法符号 X 的 FOLLOW(X)

例

计算下列文法的 FOLLOW 集:

$$E \rightarrow TE'$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$E' \rightarrow +TE' | \varepsilon$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$T \rightarrow FT'$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$T' \rightarrow *FT' | \varepsilon$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(F) = \{ (, id \}$$

计算文法符号 X 的 FOLLOW(X)

例

计算下列文法的 FOLLOW 集:

$$E \rightarrow TE'$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$E' \rightarrow +TE' | \varepsilon$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$T \rightarrow FT'$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FOLLOW}(T) = \{ +, \$,) \}$$

$$T' \rightarrow *FT' | \varepsilon$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FOLLOW}(T') = \{ +, \$,) \}$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(F) = \{ *, +, \$,) \}$$

$(A \rightarrow \alpha B \beta \in \mathbb{P}) \rightarrow (\text{FIRST}(\beta) \setminus \{\varepsilon\} \subseteq \text{FOLLOW}(B))。$

$(A \rightarrow aB \in \mathbb{P}) \vee (A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \varepsilon \in \text{FIRST}(\beta)) \rightarrow \text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)。$

例

计算下列文法的 FIRST 和 FOLLOW 集：

$S \rightarrow AB|bC$

$A \rightarrow \varepsilon|b$

$B \rightarrow \varepsilon|aD$

$C \rightarrow AD|b$

$D \rightarrow aS|c$

$(A \rightarrow \alpha B \beta \in \mathbb{P}) \rightarrow (\text{FIRST}(\beta) \setminus \{\varepsilon\} \subseteq \text{FOLLOW}(B))$ 。

$(A \rightarrow aB \in \mathbb{P}) \vee (A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \varepsilon \in \text{FIRST}(\beta)) \rightarrow \text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ 。

例

计算下列文法的 FIRST 和 FOLLOW 集：

$S \rightarrow AB|bC$

$\text{FIRST}(S) = \{\varepsilon, a, b\}$

$A \rightarrow \varepsilon|b$

$\text{FIRST}(A) = \{\varepsilon, b\}$

$B \rightarrow \varepsilon|aD$

$\text{FIRST}(B) = \{\varepsilon, a\}$

$C \rightarrow AD|b$

$\text{FIRST}(C) = \{a, b, c\}$

$D \rightarrow aS|c$

$\text{FIRST}(D) = \{a, b\}$

$(A \rightarrow \alpha B \beta \in \mathbb{P}) \rightarrow (\text{FIRST}(\beta) \setminus \{\varepsilon\} \subseteq \text{FOLLOW}(B))。$

$(A \rightarrow aB \in \mathbb{P}) \vee (A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \varepsilon \in \text{FIRST}(\beta)) \rightarrow \text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)。$

例

计算下列文法的 FIRST 和 FOLLOW 集：

$S \rightarrow AB|bC$

$\text{FIRST}(S) = \{\varepsilon, a, b\}$

$\text{FOLLOW}(S) = \{\$ \}$

$A \rightarrow \varepsilon|b$

$\text{FIRST}(A) = \{\varepsilon, b\}$

$\text{FOLLOW}(A) = \{\$, a, c\}$

$B \rightarrow \varepsilon|aD$

$\text{FIRST}(B) = \{\varepsilon, a\}$

$\text{FOLLOW}(B) = \{\$ \}$

$C \rightarrow AD|b$

$\text{FIRST}(C) = \{a, b, c\}$

$\text{FOLLOW}(C) = \{\$ \}$

$D \rightarrow aS|c$

$\text{FIRST}(D) = \{a, b\}$

$\text{FOLLOW}(D) = \{\$ \}$

预测分析表

Algorithm 3: 预测分析表构造算法

Input: $\mathbb{P}$, $\text{FIRST}(\cdot)$, $\text{FOLLOW}(\cdot)$

- ```
1 for $A \rightarrow \alpha \in \mathbb{P}$ do
2 if $a \in \text{FIRST}(\alpha)$ then
3 $M[A, a] \leftarrow (A \rightarrow \alpha)$;
4 end
5 if $\varepsilon \in \text{FIRST}(\alpha)$ then
6 for $b \in \text{FOLLOW}(A)$ do
7 $M[A, b] \leftarrow (A \rightarrow \alpha)$;
8 end
9 end
10 end
```
- 11 剩余空格打上错误标记;
- 12 如果一个格子里面有多于 1 条规则, 不是 LL(1) 文法, 返回错误;

# 预测分析表

## 例

计算下列文法的预测分析表：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

# 预测分析表

## 例

计算下列文法的预测分析表：

$$E \rightarrow TE'$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$E' \rightarrow +TE' | \varepsilon$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$T \rightarrow FT'$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$T' \rightarrow *FT' | \varepsilon$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(F) = \{ (, id \}$$

# 预测分析表

## 例

计算下列文法的预测分析表：

$$E \rightarrow TE'$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$E' \rightarrow +TE' | \varepsilon$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FOLLOW}(E') = \{ \$, ) \}$$

$$T \rightarrow FT'$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FOLLOW}(T) = \{ +, \$, ) \}$$

$$T' \rightarrow *FT' | \varepsilon$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FOLLOW}(T') = \{ +, \$, ) \}$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(F) = \{ *, +, \$, ) \}$$

# 预测分析表

|      | id                  | +                            | *                     | (                   | )                            | \$                           |
|------|---------------------|------------------------------|-----------------------|---------------------|------------------------------|------------------------------|
| $E$  | $E \rightarrow TE'$ |                              |                       | $E \rightarrow TE'$ |                              |                              |
| $E'$ |                     | $E' \rightarrow +TE'$        |                       |                     | $E' \rightarrow \varepsilon$ | $E' \rightarrow \varepsilon$ |
| $T$  | $T \rightarrow FT'$ |                              |                       | $T \rightarrow FT'$ |                              |                              |
| $T'$ |                     | $T' \rightarrow \varepsilon$ | $T' \rightarrow *FT'$ |                     | $T' \rightarrow \varepsilon$ | $T' \rightarrow \varepsilon$ |
| $F$  | $F \rightarrow id$  |                              |                       | $F \rightarrow (E)$ |                              |                              |

# LL(1) 文法的判定

## 定理

一个文法  $G$  是 **LL(1)** 文法的充分必要条件是：对于文法中的每一个非终结符  $A$  的任何两个不同的产生式  $A \rightarrow \alpha$  和  $A \rightarrow \beta$ （其中  $\alpha \neq \beta$ ），以下三个条件必须同时满足：

① **FIRST** 集不相交：

$$FIRST(\alpha) \cap FIRST(\beta) = \emptyset$$

# LL(1) 文法的判定

## 定理

一个文法  $G$  是 **LL(1)** 文法的充分必要条件是：对于文法中的每一个非终结符  $A$  的任何两个不同的产生式  $A \rightarrow \alpha$  和  $A \rightarrow \beta$ （其中  $\alpha \neq \beta$ ），以下三个条件必须同时满足：

- ①  $FIRST$  集不相交：

$$FIRST(\alpha) \cap FIRST(\beta) = \emptyset$$

- ②  $\varepsilon$  不同时存在  $\alpha$  和  $\beta$  两个候选式中，最多只能有一个可以推导出空串  $\varepsilon$ 。

# LL(1) 文法的判定

## 定理

一个文法  $G$  是 **LL(1)** 文法的充分必要条件是：对于文法中的每一个非终结符  $A$  的任何两个不同的产生式  $A \rightarrow \alpha$  和  $A \rightarrow \beta$ （其中  $\alpha \neq \beta$ ），以下三个条件必须同时满足：

- ①  $FIRST$  集不相交：

$$FIRST(\alpha) \cap FIRST(\beta) = \emptyset$$

- ②  $\varepsilon$  不同时存在  $\alpha$  和  $\beta$  两个候选式中，最多只能有一个可以推导出空串  $\varepsilon$ 。

- ③  $FIRST$  集与  $FOLLOW$  集不相交：

$$FIRST(\alpha) \cap FOLLOW(A) = \emptyset$$

同理，如果  $\alpha \Rightarrow^* \varepsilon$ ，则要求  $FIRST(\beta) \cap FOLLOW(A) = \emptyset$ 。

## SELECT 集

- 在实际工程中，通常引入 **\*\*SELECT 集\*\***（也称为预测集）的概念来简化上述充要条件。



## SELECT 集

- 在实际工程中，通常引入 **\*\*SELECT 集\*\***（也称为预测集）的概念来简化上述充要条件。
- 对于产生式  $A \rightarrow \alpha$ ，定义其 **SELECT 集**为：

## SELECT 集

- 在实际工程中，通常引入 **\*\*SELECT 集\*\***（也称为预测集）的概念来简化上述充要条件。
- 对于产生式  $A \rightarrow \alpha$ ，定义其 SELECT 集为：
  - 若  $\varepsilon \notin \text{FIRST}(\alpha)$ ，则  $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$

# SELECT 集

- 在实际工程中，通常引入 **\*\*SELECT 集\*\***（也称为预测集）的概念来简化上述充要条件。
- 对于产生式  $A \rightarrow \alpha$ ，定义其 SELECT 集为：
  - 若  $\varepsilon \notin \text{FIRST}(\alpha)$ ，则  $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$
  - 若  $\varepsilon \in \text{FIRST}(\alpha)$ ，则  $\text{SELECT}(A \rightarrow \alpha) = (\text{FIRST}(\alpha) \setminus \{\varepsilon\}) \cup \text{FOLLOW}(A)$

# SELECT 集

- 在实际工程中，通常引入 **\*\*SELECT 集\*\***（也称为预测集）的概念来简化上述充要条件。
- 对于产生式  $A \rightarrow \alpha$ ，定义其 SELECT 集为：
  - 若  $\varepsilon \notin \text{FIRST}(\alpha)$ ，则  $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$
  - 若  $\varepsilon \in \text{FIRST}(\alpha)$ ，则  $\text{SELECT}(A \rightarrow \alpha) = (\text{FIRST}(\alpha) \setminus \{\varepsilon\}) \cup \text{FOLLOW}(A)$
- LL(1) 文法的充要条件可简述为：

# SELECT 集

- 在实际工程中，通常引入 **\*\*SELECT 集\*\***（也称为预测集）的概念来简化上述充要条件。
- 对于产生式  $A \rightarrow \alpha$ ，定义其 SELECT 集为：
  - 若  $\varepsilon \notin \text{FIRST}(\alpha)$ ，则  $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$
  - 若  $\varepsilon \in \text{FIRST}(\alpha)$ ，则  $\text{SELECT}(A \rightarrow \alpha) = (\text{FIRST}(\alpha) \setminus \{\varepsilon\}) \cup \text{FOLLOW}(A)$
- LL(1) 文法的充要条件可简述为：

对于文法中每一个非终结符  $A$  的任何两个不同产生式  $A \rightarrow \alpha$  和  $A \rightarrow \beta$ ，它们的 **SELECT 集** 互不相交：

$$\text{SELECT}(A \rightarrow \alpha) \cap \text{SELECT}(A \rightarrow \beta) = \emptyset$$

# 表驱动的预测分析法

## Algorithm 4: 表驱动的预测分析法

**Input:** 输入串  $w$ , 文法  $\mathcal{G}$  的预测分析表  $M$

**Output:** 如果  $w \in \mathcal{L}(\mathcal{G})$ , 输出  $w$  的一个最左推导, 否则输出错误提示

```
1 $X \equiv$ 栈顶符号; ip 指向当前处理的输入符号;
2 while $X \neq \$/*$ 栈非空 $*/$ do
3 if $X = *ip$ then
4 $\text{pop}(X); ip++;$
5 else if $X \in V_T$ or $M[X, *ip]$ 是一条报错条目 then
6 错误处理;
7 else if $M[X, *ip] = X \rightarrow Y_1, Y_2, \dots, Y_k$ then
8 输出 $X \rightarrow Y_1, Y_2, \dots, Y_k;$
9 $\text{pop}(X);$
10 $\text{push}(Y_k); \text{push}(Y_{k-1}); \dots; \text{push}(Y_1);$
11 end
12 end
```

## 例

写出文法的预测分析表：

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

## 例

写出文法的预测分析表：

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

## 推论

如果冲突的是生成  $\varepsilon$  的产生式，那么可以将  $\varepsilon$  产生式丢弃。

## 例

基于上面的预测分析表，尝试解析  $(id * id) + id + id * id$ 。

## 知识点回顾：错误处理之恐慌模式 (Panic Mode)

- 核心思想：发现错误后，不断跳过输入符号，直到遇到一个同步令牌 (Synchronizing Token)。
- 同步令牌的选择：
  - 具有清晰边界意义的符号，如分号 ;、右花括号 }。
  - 当前非终结符的 FOLLOW 集合。

### 递归下降实现逻辑

- ① 打印错误消息。
- ② 循环调用 getNextToken() 直到当前 lookahead 属于同步集。
- ③ 从该点继续尝试分析。

# 目录

- ① 预备知识
- ② LL(1) 分析法
- ③ 小结



## 本讲小结

- 语法分析器的类型：自顶向下 vs 自底向上。



## 本讲小结

- 语法分析器的类型：自顶向下 vs 自底向上。
- LL(1) 文法的定义和特征。



## 本讲小结

- 语法分析器的类型：自顶向下 vs 自底向上。
- LL(1) 文法的定义和特征。
- 表驱动的预测分析法的结构和算法。

## 本讲小结

- 语法分析器的类型：自顶向下 vs 自底向上。
- LL(1) 文法的定义和特征。
- 表驱动的预测分析法的结构和算法。
- 构造 LL(1) 分析表的步骤，包括消除左递归、提取左公共因子、计算 FIRST 和 FOLLOW 集。

## 本讲小结

- 语法分析器的类型：自顶向下 vs 自底向上。
- LL(1) 文法的定义和特征。
- 表驱动的预测分析法的结构和算法。
- 构造 LL(1) 分析表的步骤，包括消除左递归、提取左公共因子、计算 FIRST 和 FOLLOW 集。
- LL(1) 文法的判定条件，及 SELECT 集的概念。